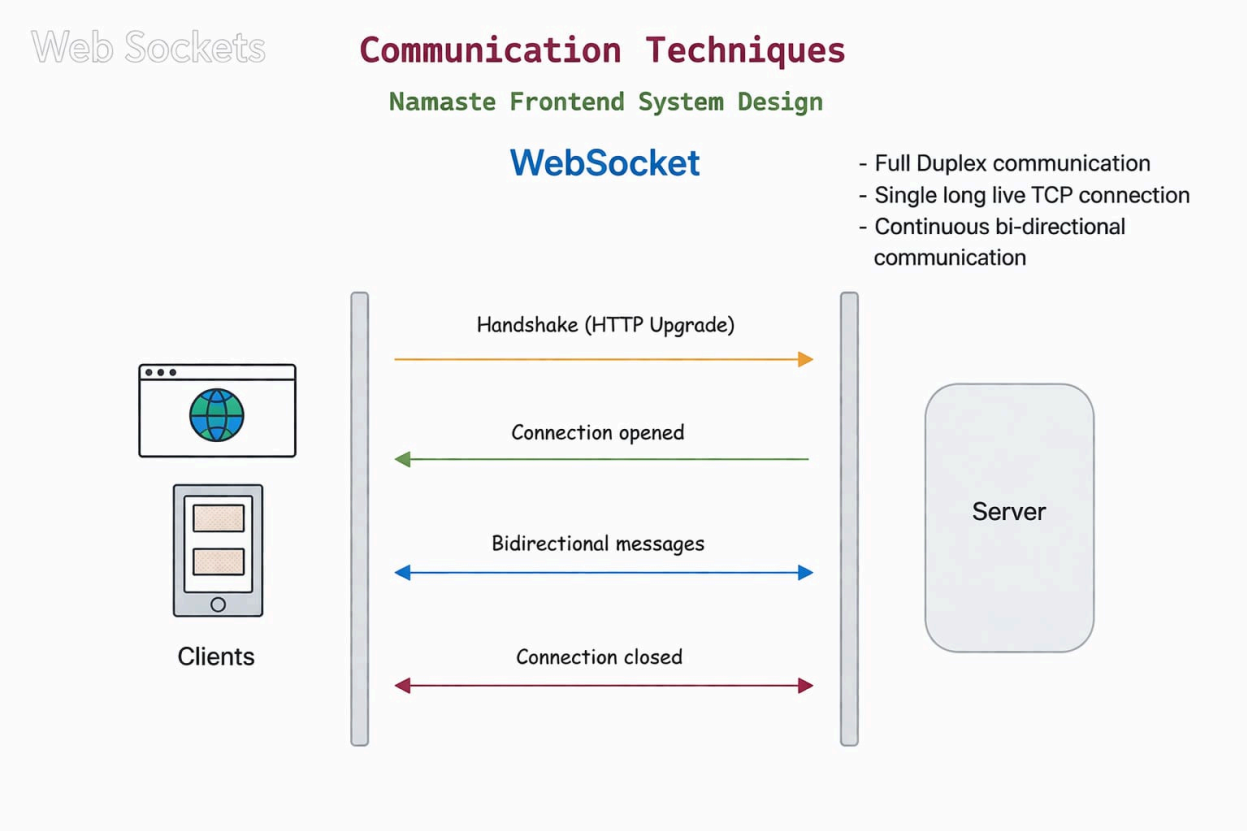
WebSockets



WebSockets provide a **persistent, full-duplex (two-way) communication channel** between client and server.

Once the connection is established, both sides can send data **at any time** without creating new HTTP requests.

Simple example

Think of it like a phone call:

- You connect once
- Both sides can talk anytime
- No need to reconnect repeatedly

How WebSocket connection is established

WebSockets begin as a normal HTTP request and then get **upgraded** to a WebSocket connection.

Step-by-step flow

1. Client sends an HTTP request:

```
GET /socket.io/?EIO=4&transport=websocket HTTP/1.1
Connection: Upgrade
Upgrade: websocket
```

1. Server responds:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
```

1. Connection is upgraded:

- No longer standard HTTP
- Becomes a persistent connection

What “Upgrade” means

- HTTP follows request → response → close
- WebSocket follows open → continuous communication

How it works after connection

1. Client connects once
2. Connection remains open
3. Client can send data anytime
4. Server can send data anytime
5. No repeated requests are required

How your code fits into this

```
const io = new Server(server);
```

```
io.on("connection", (socket) => {
```

```
socket.emit("chat-message", msg);
```

```
io.emit("chat-message", msg);
```

Advantages

- Real-time communication
- No repeated requests
- Bidirectional communication
- Efficient

Challenges in real-world systems

- Sticky sessions
- Horizontal scaling (multi-server sync)
- Connection limits
- Reconnection handling
- Message ordering and reliability
- Security concerns
- Authentication handling
- Firewalls and proxy issues
- Backward compatibility (fallback support)
- Testing and debugging becomes tough

- Resource cleanup (disconnect handling, memory leaks)

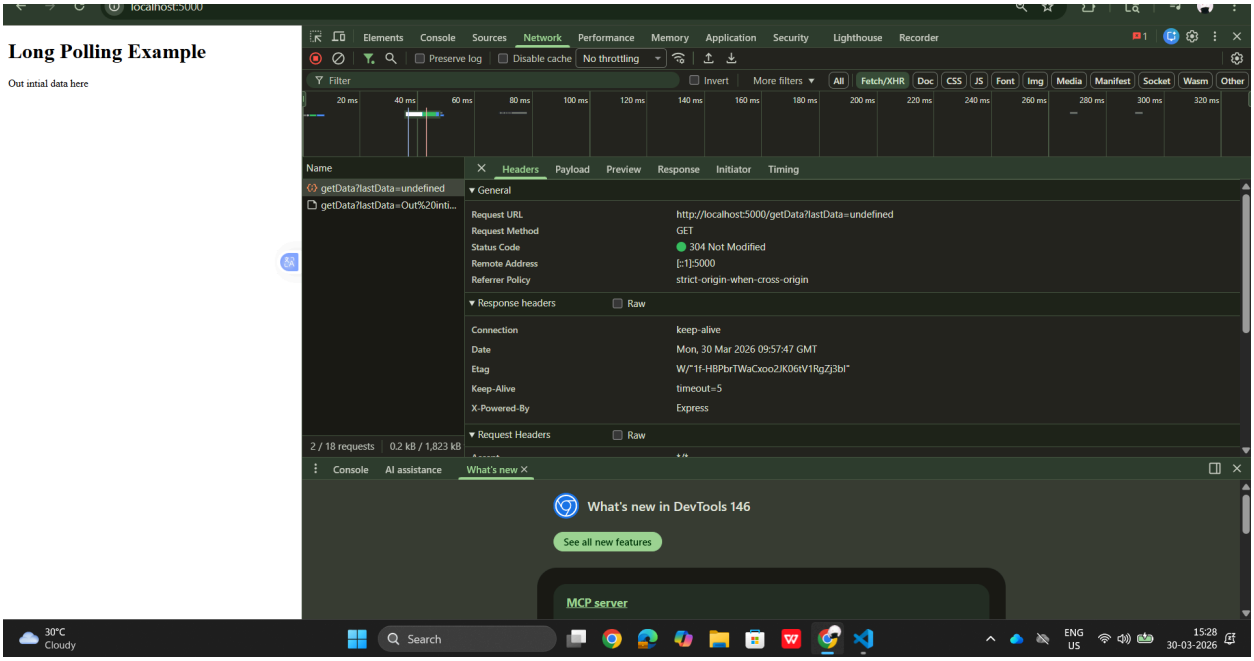
Short Polling vs Long Polling vs WebSockets

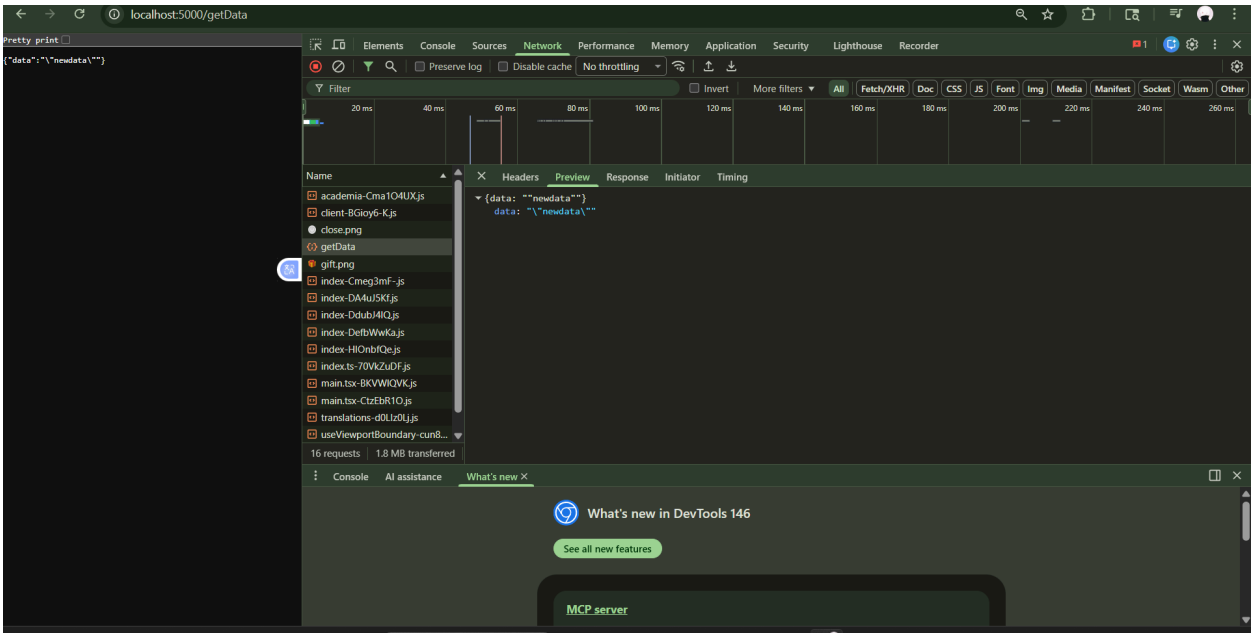
- Short polling: repeated requests
- Long polling: wait then reconnect
- WebSockets: persistent connection

Final intuition

- Short polling is simple but inefficient
- Long polling improves efficiency but still HTTP-based
- WebSockets enable real-time communication with continuous connection

Code





1. `index.html`

Code

```
<!DOCTYPE html>
<html>
<head>
  <title>Simple Chat</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
    }

    #messages {
      list-style: none;
      padding: 0;
      max-width: 500px;
    }

    #messages li {
      padding: 8px;
```

```
      border-bottom: 1px solid #ddd;
    }

    form {
      margin-top: 10px;
    }

    input {
      padding: 8px;
      width: 300px;
    }

    button {
      padding: 8px 12px;
    }
  </style>
</head>
<body>

  <h2>💬 Simple Chat</h2>

  <ul id="messages"></ul>

  <form id="form">
    <input id="input" placeholder="Type message..." autocompl
ete="off" />
    <button type="submit">Send</button>
  </form>

  <script src="/socket.io/socket.io.js"></script>

  <script>
    const socket = io();

    const form = document.getElementById("form");
    const input = document.getElementById("input");
```

```
    const messages = document.getElementById("messages");

    form.addEventListener("submit", (e) => {
      e.preventDefault();

      const msg = input.value.trim();
      if (!msg) return;

      socket.emit("chat-message", msg);
      input.value = "";
    });

    socket.on("chat-message", (msg) => {
      const li = document.createElement("li");
      li.textContent = msg;
      messages.appendChild(li);

      // auto scroll
      window.scrollTo(0, document.body.scrollHeight);
    });
  </script>

</body>
</html>
```

2. `index.js`

Code

```
const express = require("express");
const { createServer } = require("node:http");
const { join } = require("node:path");
const { Server } = require("socket.io");

const app = express();
```

```
const server = createServer(app);
const io = new Server(server);

// Serve frontend
app.get("/", (req, res) => {
  res.sendFile(join(__dirname, "index.html"));
});

// Socket connection
io.on("connection", (socket) => {
  console.log("User connected");

  socket.on("chat-message", (msg) => {
    console.log("Message:", msg);
    io.emit("chat-message", msg);
  });

  socket.on("disconnect", () => {
    console.log("User disconnected");
  });
});

// Start server
server.listen(5000, () => {
  console.log("Server running on http://localhost:5000");
});
```